

CP020003 — Artificial Intelligence 2026

Khon Kaen University

Week 6— Homework Assignment

This week, you will build a full family of recommender systems— popularity-based, user- and item-based collaborative filtering, matrix factorization, content-based filtering, and Neural Collaborative Filtering — on a dataset that already had a clean, ready-made *customer* $\times$ *book* rating matrix handed to you. This week's dataset does **not** hand you that matrix. You get a customer profile table and a product catalog table, with **no shared rating column linking the two**. Your first real job this week — before you touch a single algorithm — is to design your own interaction signal from the raw fields available. That single design decision is exactly what real recommender-system engineers spend most of their time on in practice, and it's the most important step in this assignment.

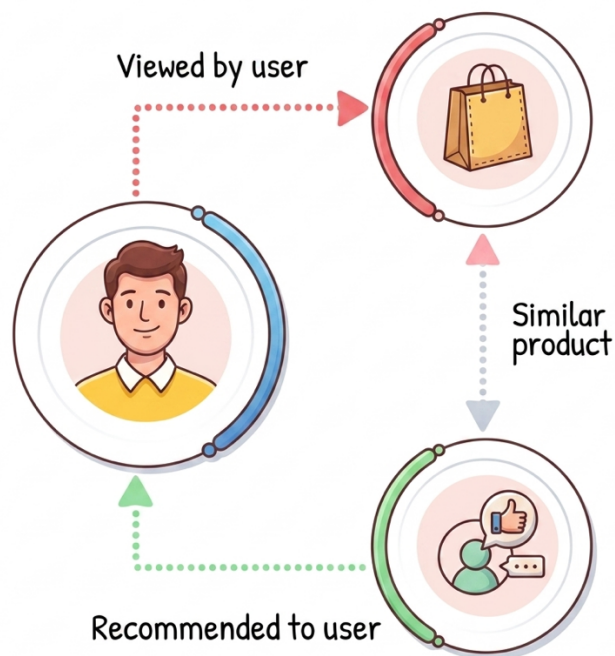
1. Dataset

Download the dataset directly from the course GitHub repository (unzip it — it contains two CSV files):

https://github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1/blob/main/dataset/e-commerce.zip

The zip extracts to an **e-commerce/** folder containing:

- customer_data_collection.csv — 10,000 customer profiles
- product_recommendation_data.csv — 10,000 products



Data Dictionary — customer_data_collection.csv

Customer_ID	Unique customer identifier, e.g. C1000.
Age / Gender / Location	Basic demographics. Location is one of 5 Indian cities (Chennai, Delhi, Bangalore, Kolkata, Mumbai).
Browsing_History	A Python-list-formatted string of product categories the customer browsed, e.g. "['Books', 'Fashion']". A weaker, higher-volume implicit signal than a purchase.
Purchase_History	A Python-list-formatted string of items the customer actually bought, e.g. "['Biography', 'Jeans']". Your strongest implicit signal — treat this like the 'explicit ratings' from last week's Book-Crossing data.
Customer_Segment	New Visitor, Occasional Shopper, or Frequent Buyer — useful for cold-start framing and business personas.
Avg_Order_Value	Average amount this customer spends per order.
Holiday / Season	Whether the record was collected during a holiday period, and the season.
Unnamed: 10	An empty trailing column from the source export — drop it during cleaning.

Data Dictionary — product_recommendation_data.csv

Product_ID	Unique product identifier, e.g. P2000.
Category / Subcategory	Category is one of 6 values (Fashion, Beauty, Electronics, Books, Home Decor, Fitness) — the same vocabulary used in customers' Browsing_History / Purchase_History, which is what makes linking the two tables possible.
Price / Brand	List price and one of 4 brand codes (Brand A–D).
Average_Rating_of_Similar_Products / Product_Rating	Average rating of comparable products in the catalog, and this specific product's own rating (roughly 1–5 scale).
Customer_Review_Sentiment_Score	A sentiment score (0–1) derived from review text.
Holiday / Season / Geographical_Location	Context fields — Geographical_Location is one of 5 countries (this is catalog metadata, not tied to any specific customer).
Similar_Product_List	A Python-list-formatted string of related product names, e.g. "['Jeans', 'Shoes']" — a ready-made item–item signal you can use directly for an item-based-style recommender, no similarity matrix required.
Probability_of_Recommendation	A 0–1 score representing how likely this product is to be recommended. Can be used as a pseudo-label for a regression-style ranking model (see the deep-learning option below).
Unnamed: 13 / Unnamed: 14	Empty trailing columns from the source export — drop them during cleaning.

2. Your Task 🎯

Suggested Problem Framing

(you may follow this OR propose your own approach, clearly justified)

Because there is no shared rating table this week, build your own **implicit customer-product interaction signal** first. A simple, defensible approach:

- Parse Browsing_History and Purchase_History (they're stored as Python-list-looking strings — use `ast.literal_eval`, exactly like you'd parse any messy real-world column).
- For each customer, treat every product whose Category matches something in that customer's Purchase_History as a positive implicit interaction (weight = 1.0); optionally add a weaker positive interaction (e.g. weight = 0.3) for category matches coming only from Browsing_History.
- This gives you a sparse Customer × Product interaction matrix — implicit and binary/weighted rather than an explicit 1–10 rating, which is a deliberately different flavor of the sparsity problem from Section 4–5 of the in-class notebook. Explain, in your own words, how implicit feedback like this differs from the explicit Book-Rating column we used in class.

Then **select at least 2 of the following techniques** (all taught hands-on in the Week 6 in-class notebook), apply each to your interaction signal, and compare them head-to-head:

- Popularity-Based — rank products using Product_Rating / Probability_of_Recommendation (a Bayesian-average-style score, as in Section 7).
- User-Based or Item-Based Collaborative Filtering — cosine similarity on your Customer × Product matrix (Sections 8–9). Item-based similarity can also be cross-checked against the ready-made Similar_Product_List column.
- Matrix Factorization (SVD) — latent factors over your interaction matrix, mean-centered as in Section 10.
- Content-Based Filtering — TF-IDF or a simple feature vector over Category / Subcategory / Brand, matched against each customer's preferred categories (Section 11) — this is a strong option here since it needs no interaction matrix at all and handles new products immediately.
- Deep Learning — since there's no shared user/item ID space to embed the way Section 12 did, build a small feed-forward network that takes customer features + product features as input and predicts Probability_of_Recommendation as a pseudo-rating target. This is the same idea as an industry ranking-stage model (Section 15) — just without ID embeddings.

Required Steps

(mirrors our in-class notebook)

1. EDA & cleaning — parse the list-formatted columns, drop the empty Unnamed columns, check for missing values, and plot the distribution of Customer_Segment, Category, Price, and Product_Rating.
2. Build your interaction signal / target — construct the Customer × Product implicit matrix (or your own justified alternative). This is the most important design decision this week — document your reasoning.
3. Pick and justify your ≥ 2 techniques — briefly explain, using the family-tree table from Section 2 of class, why each technique is a reasonable fit for this dataset's structure.
4. Build & train each chosen technique on the same train/test split (mirror the leakage-avoidance approach from Section 5–6 of class: split before you build any matrix or similarity table).

5. Generate top-N recommendations for at least 3 example customers, one from each Customer_Segment (New Visitor, Occasional Shopper, Frequent Buyer), for every technique you built.
6. Evaluate & compare — report Precision@K, Recall@K, and Coverage for each technique (Section 14 style), or RMSE if your chosen technique predicts Probability_of_Recommendation directly. Present your ≥ 2 techniques side by side in one comparison table, exactly like Section 13.
7. Handle cold start explicitly — show what your system recommends for a brand-new customer with an empty Purchase_History, and for a brand-new product with no interactions yet (Sections 7 & 11's fallback logic).
8. Business interpretation — for each Customer_Segment, state in plain language which technique you'd deploy and why, backed by your actual Precision@K/Recall@K/Coverage numbers.
9. Quantify business impact — pick your best-performing technique and estimate incremental revenue from one personalized campaign, with clearly labeled assumptions (conversion rate, average order value), exactly like the worked example in Section 17.
10. Conclusion — which technique won overall, what trade-off did you accept to get there (e.g. accuracy vs. coverage, or training cost vs. quality), and which technique would you actually ship to production first, and why?

⚠ Grading Emphasis — Read This Carefully

Getting one recommender to run is necessary, but it is not sufficient to score well this week. Most of the marks for this assignment are earned in Steps 6–9 above: how rigorously you compare your ≥ 2 techniques on the same held-out test data, and how clearly you can explain, in plain business language backed by your own Precision@K/Recall@K/Coverage table, which technique you'd actually deploy for which customer segment.

A submission with two simple techniques (e.g. Popularity vs. Content-Based) but a rigorous, honest, well-argued comparison and a clear deployment recommendation will score higher than a submission with five techniques bolted on but only a single vague sentence of comparison at the end.

3. Bonus Points ✨

- Build a third technique beyond your required two (e.g. add Matrix Factorization if your two were Popularity + Content-Based) and extend your comparison table — +bonus
- Use the Similar_Product_List column to build a lightweight item–item graph recommender (walk from a customer's purchased items to their listed "similar products") and compare it against your cosine-similarity item-based CF — a hands-on taste of the Graph Neural Network idea from Section 15 — +bonus
- Build the small feed-forward "ranking" network described above (customer features + product features $\rightarrow$ Probability_of_Recommendation) even if it isn't one of your required two techniques, and discuss how it relates to a real two-tower retrieval + ranking pipeline — +bonus
- Compare two different ways of building the implicit interaction signal (e.g. Purchase_History-only vs. Purchase_History + weighted Browsing_History) on the same technique, and show concretely how the recommendations and the evaluation numbers change — +bonus
- Quantify the cold-start business cost: estimate how many customers in the dataset are effectively "cold" (very short Purchase_History) and what fraction of total customers your fallback strategy needs to cover — +bonus